

ThinkTac Technical Assessment – Part 2: Written Explanation

Candidate: Aashritha M B

Project: SlotWise (Appointment Booking System)

For this project, I organized the code using Angular's standalone component architecture and separated responsibilities into components, services, models, and route guards. Components are responsible for the user interface, while services handle communication with Firebase Authentication and Firestore. This structure keeps the code easier to understand, maintain, and extend. I also used route guards to ensure that only authenticated users can access administrative features such as slot and booking management, while allowing customers to access the booking page without logging in.

If this application were deployed for around 10,000 users per day, I would focus on improving scalability, security, and performance. I would implement pagination for bookings instead of loading all records at once, add Firestore indexes for frequently queried fields, and optimize database reads to reduce costs. I would also move sensitive business logic, such as booking validation and confirmation, to Firebase Cloud Functions instead of relying only on the client. Monitoring, logging, automated backups, and stronger security rules would also be important for a production environment.

One feature I intentionally left out because of time constraints was an analytics and reporting dashboard. Given more time, I would add charts showing booking trends, slot utilization, and booking status distribution. I would also provide filtering, searching, and the ability to export booking data as PDF or CSV reports. These features would make the application more useful for administrators and business owners.

Apart from Angular and Firebase, I mainly used the Firebase SDK for Authentication and Firestore because they provide reliable authentication, real-time database synchronization, and simple integration with Angular. I also used Angular's reactive programming features through RxJS to handle real-time updates efficiently. These tools helped keep the application responsive while reducing the amount of custom code required.